

TUGAS 02 - EKSTRAKSI INFORMASI

Uji Coba Rule-Based Information Extraction Sederhana

Identitas Mahasiswa:

- Nama: Ahmad Hadi Nuryani
- NIM: 240411100135
- Objek: Putusan Pengadilan Negeri Blitar (PN Blitar)

```
In [4]: # Import modul os untuk manajemen folder dan membaca daftar file di sistem operasi
import os

# Import modul re (Regular Expression) sebagai mesin utama pencocokan pola aturan t
import re

# Import modul csv untuk menyimpan data hasil ekstraksi ke file CSV berbentuk tabel
import csv

# Import PyMuPDF (fitz) untuk membaca dan mengekstrak isi teks dari file dokumen PD
import fitz

# Import pandas untuk menampilkan tabel hasil ekstraksi secara rapi di notebook
import pandas as pd

print("Semua library berhasil dimuat dengan sukses!")
```

Semua library berhasil dimuat dengan sukses!

2. Konversi Dokumen Putusan dari PDF ke Teks (TXT)

Tahap ini bertujuan untuk mengonversi 5 dokumen putusan pengadilan format PDF dari PN Blitar menjadi file teks mentah (.txt). Proses ekstraksi teks per halaman menggunakan pustaka PyMuPDF (fitz) agar format teks asli di dalam berkas putusan tetap terjaga.

```
In [6]: def convert_pdf_to_txt(pdf_path, txt_path):
        """
        Fungsi untuk mengonversi satu file dokumen PDF menjadi file teks mentah (.txt).
        Parameter:
            - pdf_path : Path lengkap lokasi file PDF yang akan dibaca
            - txt_path : Path tujuan tempat menyimpan file TXT hasil konversi
        """

        # Membuka dokumen PDF menggunakan library fitz (PyMuPDF)
        doc = fitz.open(pdf_path)
        full_text = []
```

```

# Looping membaca isi teks dari setiap halaman PDF secara berurutan
for page_num in range(len(doc)):
    page = doc[page_num]
    full_text.append(page.get_text())
doc.close()

# Menggabungkan seluruh teks halaman dan menyimpannya ke file TXT (encoding UTF
with open(txt_path, 'w', encoding='utf-8') as f:
    f.write('\n'.join(full_text))

return txt_path

# --- Eksekusi Batch Konversi 5 Dokumen Putusan PN Blitar ---

# Menentukan folder sumber PDF dan folder tujuan TXT
folder_pdf = "./INPUT_PDF/"
folder_txt = "./INPUT_TXT/"
os.makedirs(folder_txt, exist_ok=True)

# Mengambil daftar seluruh file PDF di folder INPUT_PDF
daftar_file_pdf = sorted([f for f in os.listdir(folder_pdf) if f.endswith('.pdf')])

print(f"Memulai proses konversi {len(daftar_file_pdf)} dokumen PDF ke format TXT...")

# Looping mengonversi setiap file PDF menjadi file TXT
for file_pdf in daftar_file_pdf:
    path_asal = os.path.join(folder_pdf, file_pdf)
    nama_txt = file_pdf.replace('.pdf', '.txt')
    path_tujuan = os.path.join(folder_txt, nama_txt)

    # Panggil fungsi konversi
    convert_pdf_to_txt(path_asal, path_tujuan)

    # Hitung ukuran file hasil dalam Kilobyte (KB)
    ukuran_kb = os.path.getsize(path_tujuan) / 1024
    print(f"[Berhasil] {file_pdf} -> {nama_txt} ({ukuran_kb:.2f} KB)")

print("\nSeluruh dokumen putusan berhasil dikonversi ke format teks TXT!")

```

Memulai proses konversi 5 dokumen PDF ke format TXT...

```

[Berhasil] putusan_207_pid.sus_2022_pn_blt_20261001203703.pdf -> putusan_207_pid.sus_2022_pn_blt_20261001203703.txt (75.46 KB)
[Berhasil] putusan_313_pid.c_2020_pn_blt_20261001203239.pdf -> putusan_313_pid.c_2020_pn_blt_20261001203239.txt (5.78 KB)
[Berhasil] putusan_32_pid.b_2026_pn_blt_20261001202728.pdf -> putusan_32_pid.b_2026_pn_blt_20261001202728.txt (44.51 KB)
[Berhasil] putusan_34_pid.c_2026_pn_blt_20261001015134.pdf -> putusan_34_pid.c_2026_pn_blt_20261001015134.txt (18.71 KB)
[Berhasil] putusan_62_pid.b_2026_pn_blt_20261001014143.pdf -> putusan_62_pid.b_2026_pn_blt_20261001014143.txt (68.28 KB)

```

Seluruh dokumen putusan berhasil dikonversi ke format teks TXT!

3. Implementasi Mesin Rule-Based Information Extraction (Regex)

Fungsi `generateEntity(folderData, fileName)` di bawah ini merupakan mesin ekstraksi informasi berbasis aturan (*Rule-Based IE*). Sistem membaca isi dokumen teks putusan, lalu mencocokkan pola teks (*pattern matching*) menggunakan modul `re` (Regular Expression) untuk mengekstrak 15 atribut entitas perkara pengadilan.

```
In [7]: def clean_whitespace(text):
        """Fungsi pembantu untuk membersihkan spasi ganda, enter ganda, dan tabulasi."""
        if not text:
            return ""
        text = re.sub(r'[\r\n\t]+', ' ', text)
        text = re.sub(r'\s+', ' ', text)
        return text.strip()

def generateEntity(folderData, fileName):
    """
    Fungsi Rule-Based IE untuk mengekstrak 15 atribut entitas dari file teks putusan
    Parameter:
        - folderData : Path folder lokasi file TXT berada
        - fileName    : Nama file TXT yang akan diekstrak
    Return:
        - Dictionary berisi 15 atribut entitas perkara
    """
    path_file = os.path.join(folderData, fileName)
    with open(path_file, 'r', encoding='utf-8', errors='ignore') as f:
        text = f.read()

    hasil_entitas = {}

    # =====
    # 1. ENTITAS: NOMOR PUTUSAN
    # Pola: [nomorurut]/[kode perkara: Pid.B / Pid.Sus / Pid.C]/[tahun]/PN Blt
    # =====
    m_nomor = re.search(r'([0-9]+\s*/\s*[A-Za-z0-9\.\-]+\s*/\s*[0-9]{4}\s*/\s*PN(?:
    if not m_nomor:
        m_nomor = re.search(r'Nomor\s*[:\.\-]?s*([0-9]+/[A-Za-z0-9\.\-]+/[0-9]{4}/[A
    hasil_entitas['Nomor Putusan'] = clean_whitespace(m_nomor.group(1)) if m_nomor

    # =====
    # 2. ENTITAS: IDENTITAS TERDAKWA (Terdakwa 1, 2, 3)
    # Pola: Bagian tabel identitas perkara "Nama Lengkap : [Nama Terdakwa]"
    # =====
    list_terdakwa = []
    m_terdakwa = re.findall(r'(?:(Nama\s+lengkap|Nama\s+terdakwa|Nama\s+Lengkap))\s*[
    for t in m_terdakwa:
        nama_bersih = clean_whitespace(t).rstrip(';.,')
        if nama_bersih and len(nama_bersih) > 2 and nama_bersih.lower() not in [x.1
            list_terdakwa.append(nama_bersih)
```

```

# Pola cadangan jika format tabel identitas tidak formal
if not list_terdakwa:
    m_alt = re.findall(r'perkara\s+Terdakwa\s*[:\.]?\s*(?:1\.\s*)?([A-Z\s\.,]{3}
    for t in m_alt:
        ct = clean_whitespace(t).rstrip(';.,')
        if ct and ct.lower() not in [x.lower() for x in list_terdakwa]:
            list_terdakwa.append(ct)

hasil_entitas['Nama Terdakwa 1'] = list_terdakwa[0] if len(list_terdakwa) > 0 else ''
hasil_entitas['Nama Terdakwa 2'] = list_terdakwa[1] if len(list_terdakwa) > 1 else ''
hasil_entitas['Nama Terdakwa 3'] = list_terdakwa[2] if len(list_terdakwa) > 2 else ''

# =====
# 3. ENTITAS: TUNTUTAN JAKSA PENUNTUT UMUM
# Mengambil bagian isi surat tuntutan pidana yang diajukan Penuntut Umum
# =====
hasil_entitas['Tuntutan Pidana'] = ""
hasil_entitas['Tuntutan KUHP'] = ""
hasil_entitas['Tuntutan Hukuman'] = ""

m_tuntutan_sec = re.search(r'(:tuntutan\s+pidana|menuntut\s+supaya|surat\s+tun
sec_tuntutan = m_tuntutan_sec.group(1) if m_tuntutan_sec else text

# a. Tindak Pidana yang Dituntutkan
m_tp_tuntutan = re.search(r'bersalah\s+melakukan\s+tindak\s+pidana\s+["'\s\`]?([^\s
if m_tp_tuntutan:
    hasil_entitas['Tuntutan Pidana'] = clean_whitespace(m_tp_tuntutan.group(1))

# b. Pasal KUHP / UU yang Dituntutkan
m_kuhp_tuntutan = re.search(r'((?:Pasal\s+[\d\w\s\(\)]+)\s+(?:KUHP|KUHPidana|UU
if m_kuhp_tuntutan:
    hasil_entitas['Tuntutan KUHP'] = clean_whitespace(m_kuhp_tuntutan.group(1))

# c. Lama Hukuman Penjara yang Diminta Jaksa
m_huk_tuntutan = re.search(r'(:pidana\s+penjara\s+selama|penjara\s+selama|beru
if m_huk_tuntutan:
    hasil_entitas['Tuntutan Hukuman'] = clean_whitespace(m_huk_tuntutan.group(1))

# =====
# 4. ENTITAS: PUTUSAN / VONIS MAJELIS HAKIM (AMAR PUTUSAN)
# Mengambil bagian amar putusan setelah teks 'M E N G A D I L I : '
# =====
hasil_entitas['Putusan Pidana'] = ""
hasil_entitas['Putusan Hukuman'] = ""

m_mengadili_sec = re.search(r'(:M\s*E\s*N\s*G\s*A\s*D\s*I\s*L\s*I|MENGADILI)\s
sec_putusan = m_mengadili_sec.group(1) if m_mengadili_sec else text

# a. Tindak Pidana Putusan yang Terbukti
m_tp_putusan = re.search(r'(:bersalah\s+melakukan\s+tindak\s+pidana|melakukan\
if m_tp_putusan:
    hasil_entitas['Putusan Pidana'] = clean_whitespace(m_tp_putusan.group(1)).s

# b. Amar Vonis Hukuman (Penjara, Kurungan, atau Denda)
m_huk_putusan = re.search(r'(:pidana\s+penjara\s+selama|pidana\s+kurungan\s+se
if m_huk_putusan:

```

```

hasil_entitas['Putusan Hukuman'] = clean_whitespace(m_huk_putusan.group(1))

# =====
# 5. ENTITAS: APARATUR PERSIDANGAN (Tanggal, Hakim, Panitera, Penuntut)
# Mengambil bagian musyawarah penutup pengadilan
# =====

hasil_entitas['Tanggal Putusan'] = ""
hasil_entitas['Hakim Ketua'] = ""
hasil_entitas['Hakim Anggota1'] = ""
hasil_entitas['Hakim Anggota2'] = ""
hasil_entitas['Panitera'] = ""
hasil_entitas['Penuntut Umum'] = ""

m_penutup = re.search(r'(:Demikianlah\s+diputuskan|Demikian\s+diputuskan)(.*)'
sec_penutup = m_penutup.group(1) if m_penutup else text[-1800:]
sec_penutup_clean = clean_whitespace(sec_penutup)

# a. Tanggal Pembacaan Putusan
m_tgl = re.search(r'tanggal\s+([0-9]+\s+[A-Za-z]+\s+[0-9]{4})', sec_penutup_clean)
if m_tgl:
    hasil_entitas['Tanggal Putusan'] = m_tgl.group(1)

# b. Hakim Ketua (Majelis) atau Hakim Tunggal
m_hk = re.search(r'oleh\s+(?:kami,\s*)?([A-Za-z\s\.,]+?)\s*,\s*sebagai\s+Hakim\s'
if not m_hk:
    m_hk = re.search(r'([A-Z\s\.,]{5,50})\s*:\s*Hakim;', text, re.I)
if not m_hk:
    m_hk = re.search(r'oleh\s+([A-Za-z\s\.,]+?)\s*,\s*S?.?H?.?.?\s*(?:selaku\s'
if not m_hk:
    m_hk = re.search(r'Hakim\s*,\s*([A-Za-z\s\.,]+?)(?:Panitera|$)', sec_penutup)
hasil_entitas['Hakim Ketua'] = clean_whitespace(m_hk.group(1)).strip(';.,') if

# c. Hakim Anggota 1 & Hakim Anggota 2
m_ha = re.search(r'Hakim\s+Ketua\s*,\s*(.*?)\s*(?:masing\s*-\s*masing|dan)\s+se
if m_ha:
    ha_text = m_ha.group(1)
    parts = re.split(r'\s*,\s*(?=[A-Z][a-z]+(?:\s+[A-Z][a-z]+)*\s*,\s*S?.H)', ha_text)
    if len(parts) >= 2:
        hasil_entitas['Hakim Anggota1'] = clean_whitespace(parts[0]).strip(';.,')
        hasil_entitas['Hakim Anggota2'] = clean_whitespace(parts[1]).strip(';.,')
    else:
        parts = re.split(r'\s+dan\s+|\s*,\s*(?=[A-Z])', ha_text)
        if len(parts) >= 2:
            hasil_entitas['Hakim Anggota1'] = clean_whitespace(parts[0]).strip(
            hasil_entitas['Hakim Anggota2'] = clean_whitespace(parts[1]).strip(
        else:
            hasil_entitas['Hakim Anggota1'] = clean_whitespace(ha_text).strip('

# d. Panitera Pengganti
m_panitera = re.search(r'dibantu\s+oleh\s+([A-Za-z\s\.,]+?)\s*,\s*(?:selaku\s+)
if not m_panitera:
    m_panitera = re.search(r'([A-Z\s\.,]{5,50})\s*:\s*Panitera\s+Pengganti;', text)
if not m_panitera:
    m_panitera = re.search(r'Panitera\s+Pengganti\s*[:\.,]?\s*([A-Za-z\s\.,]+?)(
hasil_entitas['Panitera'] = clean_whitespace(m_panitera.group(1)).strip(';.,')

```

```

# e. Jaksa Penuntut Umum
m_pu = re.search(r'(?:(dihadiri|dihadiri\s+pula)\s+oleh\s+([A-Za-z\s\.,]+?)\s*,\s+
if not m_pu:
    m_pu = re.search(r'Penuntut\s+Umum\s*[:\.\.]\s*([A-Za-z\s\.,]+?)(?:dan|pada|
hasil_entitas['Penuntut Umum'] = clean_whitespace(m_pu.group(1)).strip(';.,') i

return hasil_entitas

print("Fungsi generateEntity berhasil didefinisikan dan siap digunakan!")

```

Fungsi generateEntity berhasil didefinisikan dan siap digunakan!

4. Fungsi Penyimpanan Output (File Teks Individual & Rekap CSV)

Sesuai instruksi tugas, hasil ekstraksi entitas harus disimpan dalam dua bentuk:

1. **File Teks Individual (OUTPUT/O-[namafile].txt)**: Menyimpan 15 atribut entitas per dokumen putusan.
2. **File CSV Rekapitulasi (courtHistory.csv)**: Menggabungkan seluruh hasil ekstraksi ke dalam satu tabel CSV terstruktur dengan 15 kolom entitas.

```

In [8]: def save_individual_output(entity_dict, output_folder, file_name):
        """
        Fungsi untuk menyimpan hasil ekstraksi satu putusan ke file teks O-[namafile].t
        """
        os.makedirs(output_folder, exist_ok=True)
        output_filename = f"O-{file_name}"
        output_path = os.path.join(output_folder, output_filename)

        with open(output_path, 'w', encoding='utf-8') as f:
            f.write("=" * 60 + "\n")
            f.write(f"HASIL EKSTRAKSI ENTITAS: {file_name}\n")
            f.write("=" * 60 + "\n\n")
            for key, value in entity_dict.items():
                f.write(f"{key:<20} : {value}\n")

        return output_path

def save_to_csv(list_entities, csv_path="courtHistory.csv"):
    """
    Fungsi untuk menyimpan seluruh rekapitulasi entitas putusan ke dalam file CSV.
    Header kolom disesuaikan dengan spesifikasi tugas ekstraksi informasi.
    """
    fieldnames = [
        'Nomor Putusan', 'Nama Terdakwa 1', 'Nama Terdakwa 2', 'Nama Terdakwa 3',
        'Tuntutan Pidana', 'Tuntutan KUHP', 'Tuntutan Hukuman',
        'Putusan Pidana', 'Putusan Hukuman', 'Tanggal Putusan',
        'Hakim Ketua', 'Hakim Anggota1', 'Hakim Anggota2',
        'Panitera', 'Penuntut Umum'
    ]

```

```

with open(csv_path, 'w', newline='', encoding='utf-8') as f:
    writer = csv.DictWriter(f, fieldnames=fieldnames)
    writer.writeheader()
    for item in list_entities:
        writer.writerow(item)

return csv_path

print("Fungsi penyimpanan output (TXT & CSV) siap digunakan!")

```

Fungsi penyimpanan output (TXT & CSV) siap digunakan!

5. Eksekusi Batch Ekstraksi Informasi dari 5 Berkas Putusan PN Blitar

Pada tahap ini, sistem akan:

1. Membaca seluruh file `.txt` yang berada di direktori `INPUT_TXT/`.
2. Menjalankan fungsi `generateEntity()` untuk mengekstrak 15 entitas dari masing-masing berkas.
3. Menyimpan file individual ke folder `OUTPUT/O-[namafile].txt`.
4. Mengompilasi seluruh entitas ke dalam list dan menyimpannya ke `courtHistory.csv`.

```

In [9]: folder_input_txt = "./INPUT_TXT/"
        folder_output = "./OUTPUT/"
        csv_file_path = "courtHistory.csv"

# Mengambil daftar file TXT yang tersedia
daftar_file_txt = sorted([f for f in os.listdir(folder_input_txt) if f.endswith('.t

list_semua_hasil = []

print(f"Memulai proses ekstraksi informasi dari {len(daftar_file_txt)} berkas putus

for i, nama_file in enumerate(daftar_file_txt, 1):
    print(f"[{i}/{len(daftar_file_txt)}] Mengekstrak file: {nama_file}")

    # 1. Jalankan Rule-Based IE
    entitas = generateEntity(folder_input_txt, nama_file)
    list_semua_hasil.append(entitas)

    # 2. Simpan file individual O-[namafile].txt
    path_txt_out = save_individual_output(entitas, folder_output, nama_file)

    # Tampilkan ringkasan singkat di console
    print(f"    -> Nomor Putusan : {entitas['Nomor Putusan']}")
    print(f"    -> Terdakwa 1 : {entitas['Nama Terdakwa 1']}")
    print(f"    -> Putusan Hukuman: {entitas['Putusan Hukuman']}")
    print(f"    -> Disimpan di : {path_txt_out}\n")

# 3. Simpan seluruh data ke file CSV rekapitulasi

```

```
save_to_csv(list_semua_hasil, csv_file_path)
print(f"Ekstraksi selesai! Rekapitulasi berhasil disimpan di '{csv_file_path}'.")
```

Memulai proses ekstraksi informasi dari 5 berkas putusan PN Blitar...

```
[1/5] Mengekstrak file: putusan_207_pid.sus_2022_pn_blt_20261001203703.txt
-> Nomor Putusan : 207/Pid.Sus/2022/PN Blt
-> Terdakwa 1 : Kateni Bin Alm Jaimin
-> Putusan Hukuman: 4 (empat) Bulan dan denda sebesar Rp 3
-> Disimpan di : ./OUTPUT/0-putusan_207_pid.sus_2022_pn_blt_20261001203703.txt

[2/5] Mengekstrak file: putusan_313_pid.c_2020_pn_blt_20261001203239.txt
-> Nomor Putusan : 313/Pid.C/2020/PN Blt
-> Terdakwa 1 : Suwito
-> Putusan Hukuman: 5 (lima) hari
-> Disimpan di : ./OUTPUT/0-putusan_313_pid.c_2020_pn_blt_20261001203239.txt

[3/5] Mengekstrak file: putusan_32_pid.b_2026_pn_blt_20261001202728.txt
-> Nomor Putusan : 32/Pid.B/2026/PN Blt
-> Terdakwa 1 : Sholikhin Alias Likok Bin Ginar
-> Putusan Hukuman: 10 (sepuluh) bulan
-> Disimpan di : ./OUTPUT/0-putusan_32_pid.b_2026_pn_blt_20261001202728.txt

[4/5] Mengekstrak file: putusan_34_pid.c_2026_pn_blt_20261001015134.txt
-> Nomor Putusan : 34/Pid.C/2026/PN Blt
-> Terdakwa 1 : LANANG TRI ARDIANSYAH
-> Putusan Hukuman: Rp200
-> Disimpan di : ./OUTPUT/0-putusan_34_pid.c_2026_pn_blt_20261001015134.txt

[5/5] Mengekstrak file: putusan_62_pid.b_2026_pn_blt_20261001014143.txt
-> Nomor Putusan : 62/Pid.B/2026/PN Blt
-> Terdakwa 1 : Irwanto
-> Putusan Hukuman: 3 (tiga) tahun
-> Disimpan di : ./OUTPUT/0-putusan_62_pid.b_2026_pn_blt_20261001014143.txt
```

Ekstraksi selesai! Rekapitulasi berhasil disimpan di 'courtHistory.csv'.

6. Verifikasi dan Tampilan Tabel Rekapitulasi Data (Pandas)

Tabel berikut menampilkan isi file `courtHistory.csv` yang memuat ke-15 atribut entitas dari 5 putusan Pengadilan Negeri Blitar yang telah diekstrak menggunakan mesin *Rule-Based IE*.

```
In [10]: # Membaca file courtHistory.csv ke dalam DataFrame Pandas
df_hasil = pd.read_csv("courtHistory.csv")

# Mengatur konfigurasi tampilan pandas agar seluruh kolom terlihat jelas
pd.set_option('display.max_columns', None)
pd.set_option('display.width', 1000)

print(f"Total Dokumen Putusan yang Berhasil Diekstrak: {len(df_hasil)}\n")
```

```
# Menampilkan tabel hasil ekstraksi
df_hasil
```

Total Dokumen Putusan yang Berhasil Diekstrak: 5

Out[10]:

	Nomor Putusan	Nama Terdakwa 1	Nama Terdakwa 2	Nama Terdakwa 3	Tuntutan Pidana	Tuntutan KUHP	Tuntul Hukurr
0	207/Pid.Sus/2022/PN Blt	Kateni Bin Alm Jaimin	NaN	NaN	menganiaya dan/atau	Pasal 91 B ayat (1) jo Pasal 66 A ayat (1) UU ...	4 (emp Bulan c
1	313/Pid.C/2020/PN Blt	Suwito	NaN	NaN	mabuk ditempat umum dan mengganggu	Pasal 492 Ayat 1 KUHP	N
2	32/Pid.B/2026/PN Blt	Sholikhin Alias Likok Bin Ginar	NaN	NaN	melakukan penganiayaan	NaN	(sepul bu
3	34/Pid.C/2026/PN Blt	LANANG TRI ARDIANSYAH	NaN	NaN	mabuk di	Pasal 316 KUHP	N
4	62/Pid.B/2026/PN Blt	Irwanto	NaN	NaN	pencurian dalam keadaan memberatkan	Pasal 477 ayat (1) huruf g KUHP	3 (Ti Tahun c 6 (Ena Bul: (ti Tah

7. Kesimpulan dan Analisis Evaluasi

Berdasarkan hasil uji coba sistem *Rule-Based Information Extraction* pada 5 berkas putusan Pengadilan Negeri Blitar, dapat ditarik beberapa poin kesimpulan dan analisis:

1. Efektivitas Pendekatan Rule-Based (Regex):

- Pendekatan berbasis aturan (*pattern matching*) dengan *Regular Expression* terbukti sangat cepat dan efektif untuk mengekstrak informasi pada dokumen legal yang memiliki struktur baku, seperti nomor perkara, identitas terdakwa pada tabel identitas, dan susunan aparatur pengadilan (Hakim, Panitera, Penuntut Umum).

2. Karakteristik dan Variasi Dokumen Putusan:

- **Perkara Pidana Biasa (Pid.B) & Pidana Khusus (Pid.Sus):** Memiliki struktur persidangan lengkap (Majelis Hakim terdiri dari 1 Hakim Ketua dan 2 Hakim Anggota, dihadiri Jaksa Penuntut Umum, serta terdapat surat tuntutan dan amar vonis).
- **Perkara Tindak Pidana Ringan / Cepat (Pid.C / Tipiring):** Disidangkan oleh **Hakim Tunggal** tanpa kehadiran Jaksa Penuntut Umum dan tanpa pembacaan surat tuntutan formal. Aturan regex yang dirancang telah berhasil menangani kondisi ini tanpa mengalami *error/crash*.

3. Tantangan Format Teks:

- Pemotongan baris (*line wrap*) dan spasi tidak beraturan pada hasil konversi PDF ke teks merupakan tantangan utama. Penggunaan fungsi `clean_whitespace()` dan *null-safe regex* berhasil mencegah kegagalan ekstraksi teks.